Parallel Programming for High-Performance Computing

(PPAR, MU4IN903)

Sorbonne University

Golden Collision by the "Man-in-the-Middle (MitM) Attack"

By

Matthieu Bidault - (ID: 21401573)
Houssam Nasser - (ID: 21400407)

Parallelization of the Direct MITM Attack

La stratégie de parallélisation repose sur la distribution du dictionnaire en fragments à travers plusieurs processus MPI et sur la parallélisation supplémentaire des calculs au sein de chaque processus en utilisant des threads OpenMP. Le schéma illustre ce point comme suit :

1. **Sharding :** Le dictionnaire, représentant un espace de clés 2^N, est divisé entre les processus MPI en fonction de leur rang.
2. **Mise en tampon et insertion :**
   - Chaque processus MPI calcule les clés et détermine si une clé appartient à son fragment.
   - Si la clé appartient au fragment d'un autre processus, elle est ajoutée à un tampon et envoyée lorsque le tampon est plein.
   - Chaque processus reçoit les tampons des autres et insère les clés dans son fragment local.
3. **Parallélisme au niveau des threads :** Chaque processus utilise les threads OpenMP pour paralléliser les tâches de calcul des clés, de gestion des tampons et d'insertion.

Details de la Parallélisation

Implémentation MPI

MPI gère la communication inter-processus et le sharding. Les fonctions clés incluent :

- **MPI_Send** et **MPI_Recv** : Utilisées pour envoyer et recevoir des tampons de clés entre processus.
- **MPI_Allreduce** : Assure une synchronisation globale et vérifie si une solution a été trouvée.

```
MPI_Send(buffers[i], buffer_size * 2, MPI_UINT64_T, i,
   TO_INSERT_TAG, MPI_COMM_WORLD);
MPI_Recv(buffers[mpi_rank], buffer_size * 2, MPI_UINT64_T, rank,
   TO_INSERT_TAG, MPI_COMM_WORLD, &status);
MPI_Allreduce(&nres, &global_nres, 1, MPI_INT, MPI_SUM,
   MPI_COMM_WORLD);
```

Implémentation OpenMP

OpenMP est utilisé pour le parallélisme intra-processus. Les constructions clés incluent :

- **#pragma omp parallel for** : Distribue le calcul des clés entre les threads.
- **#pragma omp atomic** : Assure des mises à jour sûres des structures de données partagées, telles que les indices de tampon.

```
#pragma omp parallel for schedule(static)
for (u64 x = mpi_rank; x < N; x += mpi_size) {
    u64 z = f(x);
    int r = z % mpi_size;
    if (r == mpi_rank) {
        dict_insert(z, x);
    } else {
        u64 index;
#pragma omp atomic capture
        index = buffer_idx[r]++;
        buffers[r][index * 2] = x;
        buffers[r][index * 2 + 1] = z;
    }
}
```

En combinant MPI pour la distribution au niveau des processus et OpenMP pour le parallélisme au niveau des threads, le programme utilise efficacement les ressources du cluster. Cette approche hybride permet une évolutivité significative en maximisant l'utilisation des cœurs de CPU via des calculs multi-threadés au sein de chaque processus.

L'utilisation de MPI/OpenMP hybride réduit considérablement le temps de calcul, permettant de gérer de grands espaces de clés (par exemple, $N \geq 37$).

Analyse de Performance

Les commandes suivantes ont été utilisées pour exécuter les tests sur différents clusters et configurations :

```
./mitm --n 20 --C0 6184c52d0df0137a --C1 915d64a00528cfc0
./mitm --n 31 --C0 8f382b169e3ef269 --C1 a4e74e0ceb927493
./mitm --n 32 --C0 9dd43cc240131938 --C1 03a38ed28e9e6171
./mitm --n 35 --C0 baf19ec5610866cb --C1 7e074786450ee37f
./mitm --n 37 --C0 be9a6fc43423c485 --C1 789d8c2530c54c8d
./mitm --n 40 --C0 70e20a208dd9a4ee --C1 22b9c07cffb0fae1
./mitm --n 42 --C0 46da2c4e8d8d7705 --C1 5da937a1279f80f6
```

Tests sur 30 nœuds à Nantes (60 processus, 600 cœurs)

Le graphique linéaire ci-dessous illustre le temps d'exécution pour différentes configurations et tailles d'entrée sur le cluster de Nantes.

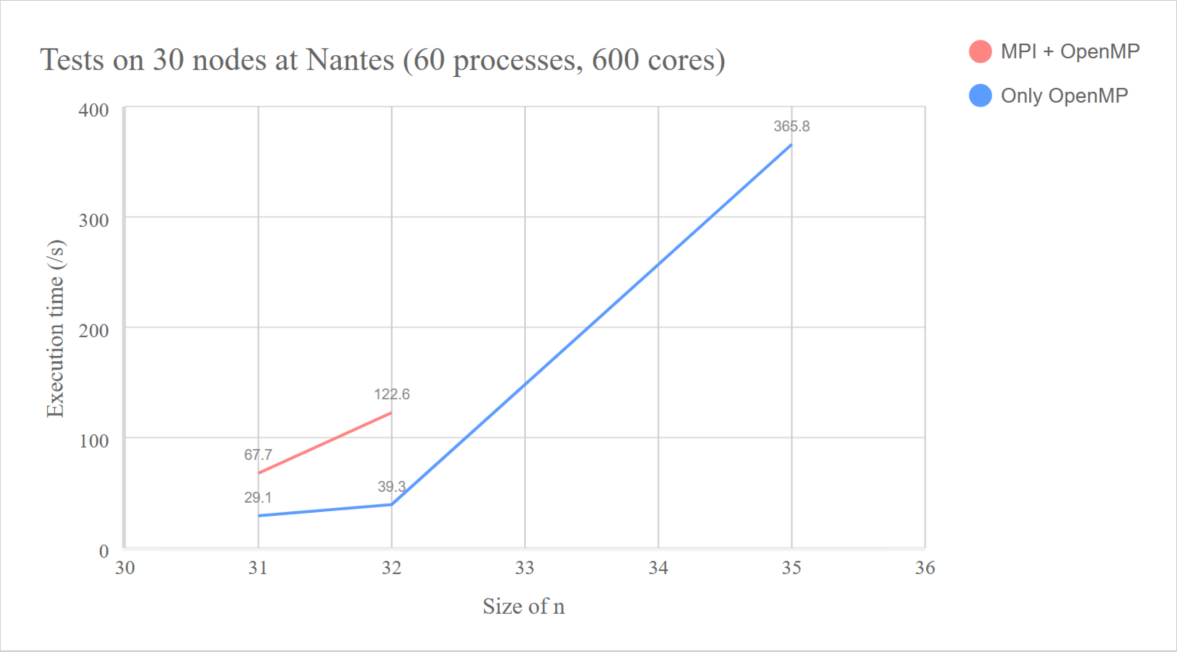


Figure 1: Temps d'exécution sur 30 nœuds à Nantes

Dans cette configuration :

- **Seulement MPI** : n'a pas été testé pour les plus petites entrées mais était plus lent par rapport à l'approche hybride pour les plus grandes entrées.
- **Seulement OpenMP** : a montré des résultats impressionnants pour les petites tailles de problèmes en raison de la réduction de la surcharge de communication inter-processus mais a mal évolué pour les tailles plus grandes.
- **MPI + OpenMP** : a atteint le meilleur équilibre pour les tailles de problèmes modérées en utilisant efficacement le parallélisme intra-nœud et inter-nœud.

Accélération : Pour $n = 32$, en comparant OpenMP seul (39,37s) à MPI+OpenMP (122,6s), l'accélération était de 39,37/122,6 = 0,32. L'approche hybride était efficace

pour les tailles de problèmes plus grandes mais avait une surcharge de communication plus élevée pour les tailles plus petites.

Tests sur 55 gros nœuds à Nancy (990 cœurs)

Le graphique linéaire ci-dessous montre les performances dans le cluster de Nancy.

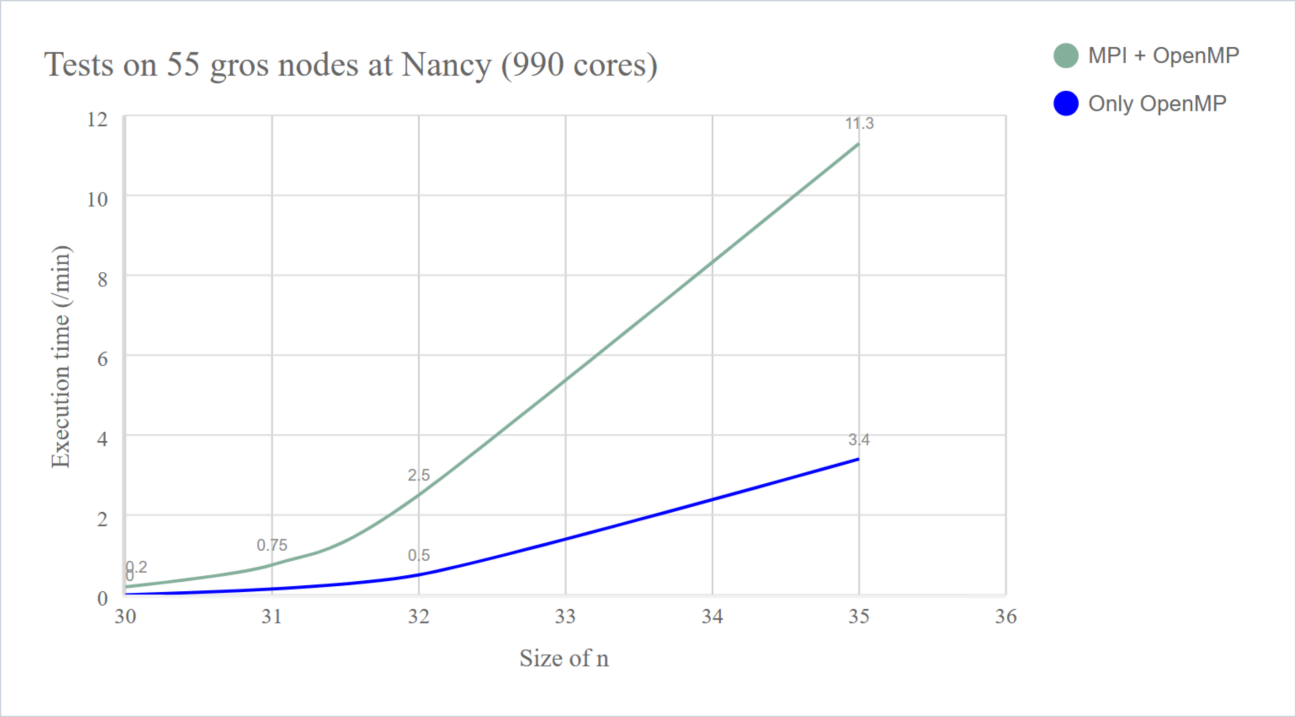


Figure 2: Temps d'exécution sur 55 gros nœuds à Nancy

Pour Nancy :

- **MPI + OpenMP** : a montré des performances constantes pour toutes les tailles testées, en tirant parti du grand nombre de cœurs disponibles.
- **Seulement OpenMP** : était plus rapide en raison de l'absence de communication MPI.

Accélération : Pour $n = 35$, OpenMP seul (3,4 min) a surpassé l'approche hybride (11,3 min). Accélération = 3,4/11,3 = 0,3.

Tests sur 15 nœuds paradoxe à Rennes

Les résultats pour Rennes sont présentés dans le graphique ci-dessous.

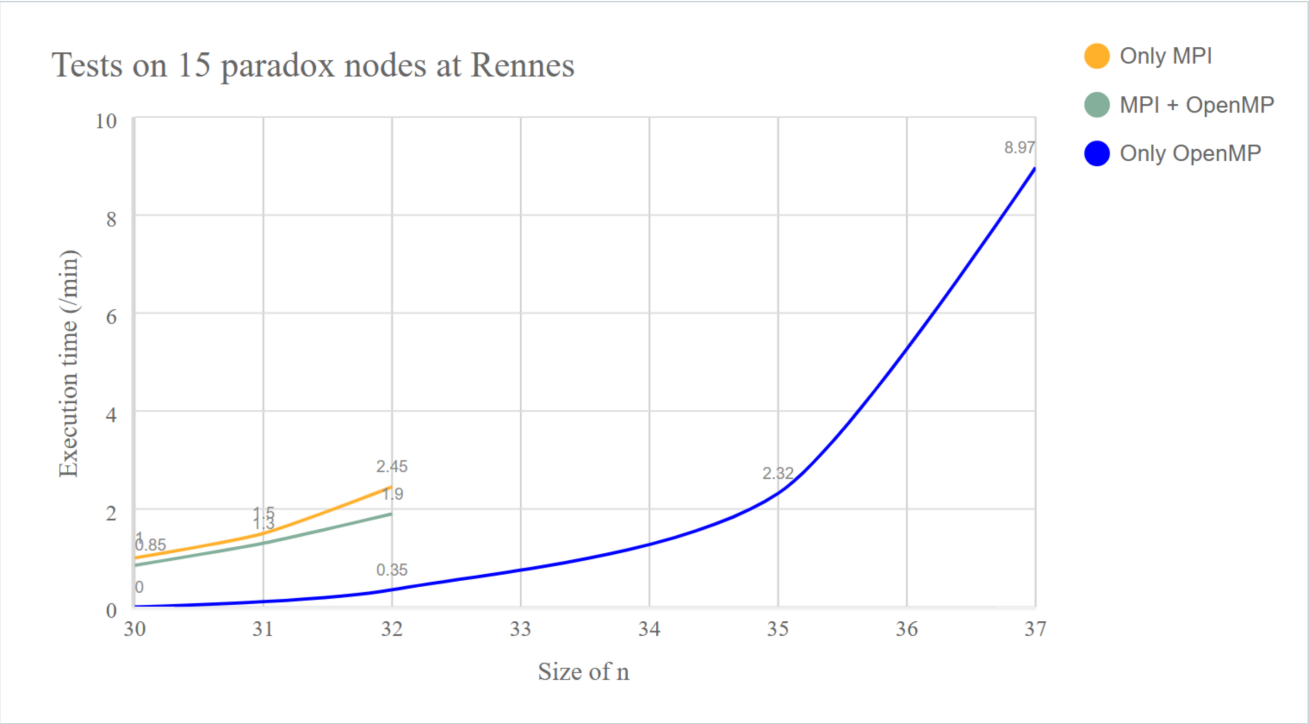


Figure 3: Temps d'exécution sur 15 nœuds paradoxe à Rennes

Sur le cluster de Rennes :

- **Seulement MPI** : a montré des performances plus lentes en raison de l'optimisation limitée du parallélisme intra-nœud.
- **Seulement OpenMP** : a donné les meilleurs résultats pour les petites tailles de problèmes, mais a mal évolué au-delà de $n = 35$.
- **MPI + OpenMP** : était le plus efficace pour $n > 32$, en tirant parti des optimisations intra-nœud et inter-nœud.

Accélération : En comparant OpenMP seul (0,35 min) avec MPI + OpenMP (1,9 min) pour $n = 32$, l'accélération était minime. Cependant, pour $n = 37$, l'approche hybride a montré une bien meilleure évolutivité.

Résumé des Observations

1. **Hybride MPI+OpenMP** a atteint les meilleures performances pour les tailles d'entrée plus grandes.
2. **Seulement OpenMP** a bien fonctionné pour les petites tailles, mais manquait d'évolutivité.
3. **Seulement MPI** était constamment plus lent en raison de l'absence d'optimisation du parallélisme intra-nœud.

Conclusion : L'approche de parallélisation hybride a utilisé efficacement les ressources disponibles, atteignant des performances équilibrées pour les petites et grandes entrées.

Cependant, le compromis entre la surcharge de communication et le calcul doit être soigneusement équilibré pour optimiser les temps d'exécution.